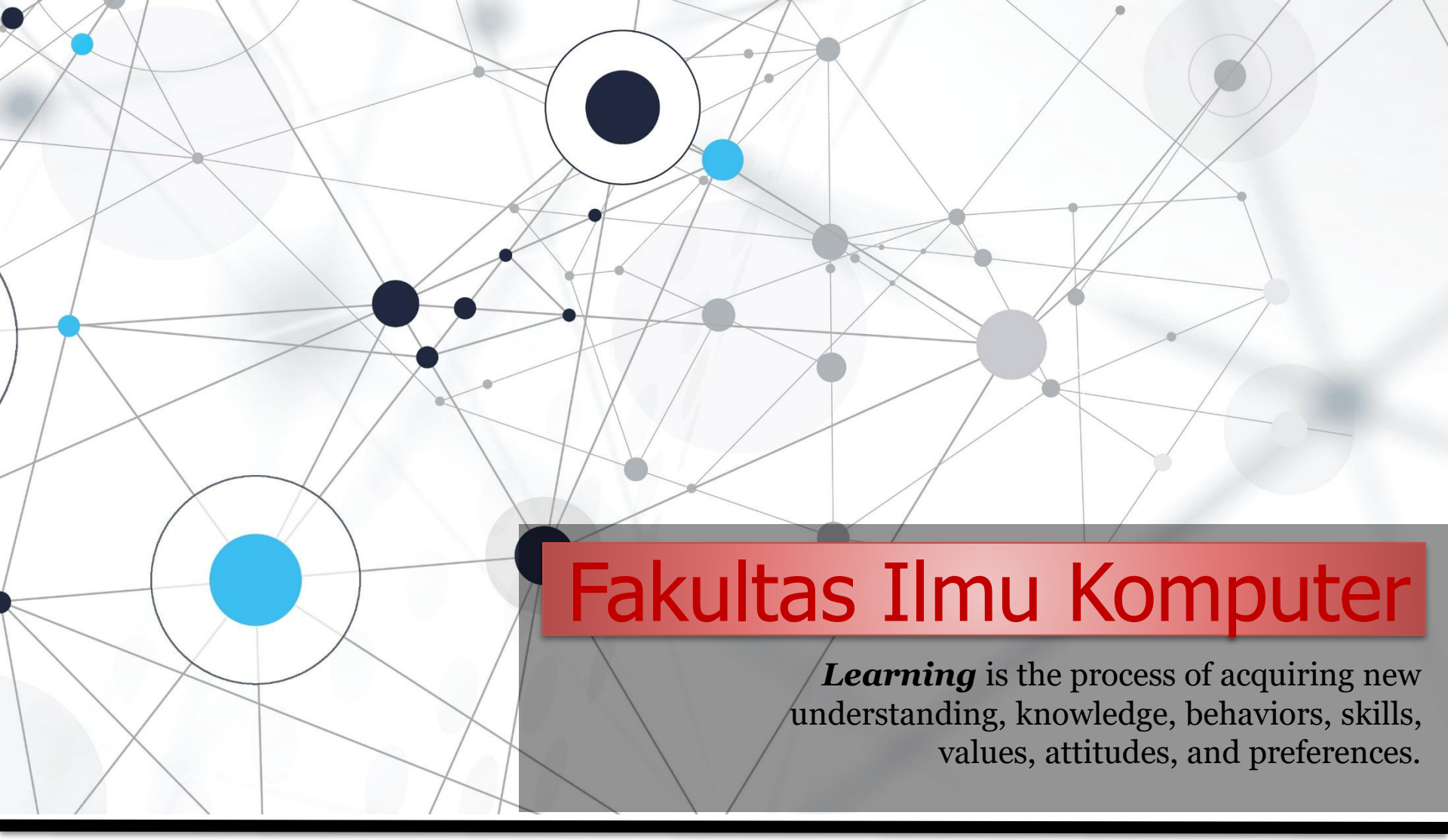


Quiz #1

(Object Oriented Programming)

Question #1

What is a OOP in programming?

A. **Object-oriented programming (OOP) is a computer programming model that we can use to generate word vectors of feature data in machine learning algorithms and is used to organize software design around data, or objects, rather than functions and logic.**

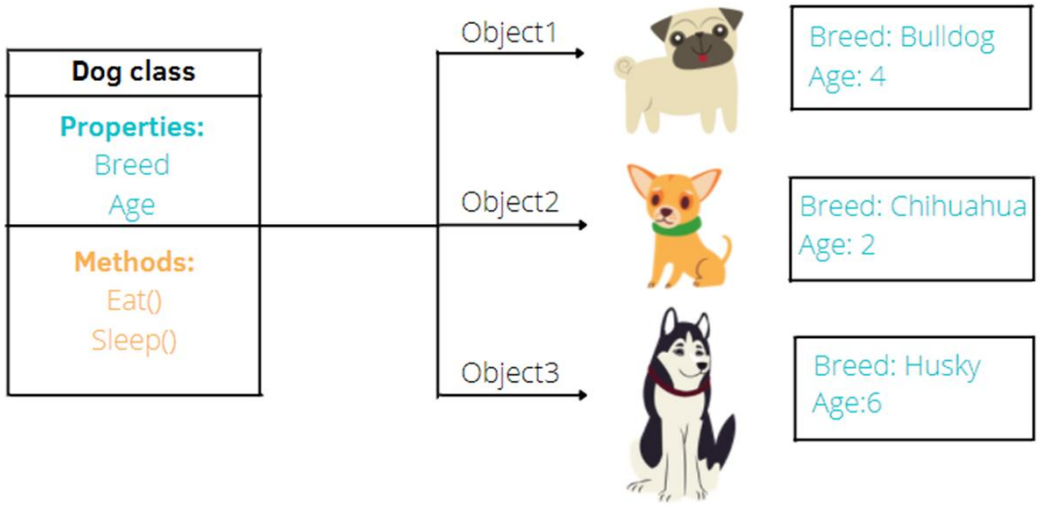
B. **Object-oriented programming (OOP) is a *compiler type* of programming characterized by the identification of classes of objects closely linked with the methods (*functions*) with which they are associated.**

C. **Object-oriented programming (OOP) is a method of structuring a program by bundling related properties and behaviors into individual peace of strawberry by identifying classes of objects that are closely related to the methods which are associated.**

D. **Object-Oriented Programming (OOP) is a programming paradigm in computer science that relies on the concept of classes and objects. It is used to structure a software program into simple, reusable pieces of code blueprints (usually called classes), which are used to create individual instances of objects.**

E. **The object-oriented programming is basically a computer user interface design philosophy or methodology that organizes/models software design around data or objects rather than functions and logic.**

Question #2



Perhatikan gambar di samping dan pilihlah *statement* **yang salah** pada opsi di bawah ini.

A. **Terdapat tiga objects yang dibuat dari class Dog.**

B. **Class Dog mewariskan semua properties/variables dan methods ke class Object1, Object2 dan Object3.**

C. **Object1, Object2 dan Object3 mempunyai properties/variables yang berbeda-beda.**

D. **Class Dog mempunyai dua variables dengan nama Breed dan Age.**

E. **Class Dog dapat mengakses dua method yaitu Eat() method dan Sleep() method.**

Question #3

Access specifiers atau *access modifiers* di dalam Python programming digunakan untuk membatasi akses ke *variabel kelas* dan *metode kelas* dari luar kelas sambil menerapkan konsep pewarisan. Pilihlah **yang bukan** merupakan *access modifiers* di bahasa pemrograman Python ???

A. **Scope access modifier.**

B. **Public access modifier.**

C. **Protected access modifier.**

D. **All the variables and methods in Python are by default public.**

E. **Private access modifier.**

Question #4

```
# Parent Class
class PersonSkill:
    def teach(self):
        print("Method 1")

    def run(self):
        print("Method 2")

    def see(self):
        print("Method 3")

    def __sleep(self):
        print("Method 4")

    def __speak(self):
        print("Method 5")

# Child Class #1
class Teacher(Personskill):
    def teach(self):
        print("I'm teaching !!!");

# Child Class #2
class Worker(PersonSkill):
    def analyze(self):
        print("I'm working !!!");

# create objects
obj1 = Teacher()
obj2 = Worker()
```

Perhatikan dengan teliti pada proses pewarisan di samping. Jika *object* dari *class* Teacher dibuat, maka *object* tersebut tidak dapat mengakses berapa method?

A. 2 Method

B. 3 Method

C. 4 Method

D. 5 Method

E. NameError: name 'Personskill' is not defined.

Question #5

Demo2.py

```python
# create class
class Student:

    # Instance attribute in method constructor (default method)
    def __init__(self, name):
        self.name = name;

    # method #1 (create a function)
    def info(self):
        print("My name is {}.".format(self.name))

# main file
if __name__ == "__main__":
    # create objects
    Rodger = Student("Rodger Patty")
    Tommy = Student("Tommy Modoaggo")
    Semmy = Student("Semmy Bambo")
    Julius = Student("Julius Meisi")
    Paksi = Student("Paksi Tegar")
    Putri = Student("Putri Mamarimbing")

    # Accessing class methods
    Julius.info()
```

Perhatikan code program di samping, pilihlah *output* yang tepat dari code program tersebut.

A. **TypeError: Employee() takes no arguments.**

B. **My name is Julius Meisi.**

C. **SyntaxError: invalid syntax.**

D. **TypeError: __init__() missing 1 required positional argument: “name”.**

E. **My name is Semmy Bambo.**

Question #6

Perhatikan code program di samping, pilihlah *output* yang tepat dari code program tersebut.

```python
# create parent class
class Bird:
    # variables
    name = "Burung Gereja"
    color = "Merah";

    # method
    def info(self):
        print("Nama: {0} dan warna: {1}.".format(self.name, self.color))

# create child class 1
class Eagle(Bird):
    # constructor
    def __init__(self, nama, warna):
        self.name = nama
        self.color = warna


# create child class 2
class Weka(Bird):
    # constructor
    def __init__(self, nama, warna):
        self.name = nama
        self.color = warna

# # object of parent class & child class created
obj_burung = Bird()
obj_elang = Eagle("Elang", "Coklat")
obj_weka = Weka("Weka", "Hitam")

# user-defined function is called from obj_burung (parent class)
obj_burung.info();
obj_elang.info();
obj_weka.info();
```

A. **TypeError: Bird() takes no arguments.**

B. **Nama: Burung Gereja dan warna: Merah.**
Nama: Burung Gereja dan warna: Merah.
Nama: Burung Gereja dan warna: Merah.

C. **TypeError: __init__() missing 1 required positional argument: “name”.**

D. **Nama: Burung Gereja dan warna: Merah.**
Nama: Elang dan warna: Coklat.
Nama: Weka dan warna: Hitam.

E. **Nama: Elang dan warna: Coklat.**
Nama: Elang dan warna: Coklat.
Nama: Weka dan warna: Hitam.

Question #7

```python
class Animal:
    # Constructor
    def __init__(self):
        print("A")
        self.x = 10

    # Destructor
    def __del__(self):
        print("B")

# create objects
obj1 = Animal()  # Creating object 1
obj2 = obj1      # Creating object 2
obj3 = Animal()  # Creating object 3
obj4 = obj3      # Creating object 4
```

Perhatikan penerapan constructor dan destructor pada class Animal di samping. Apa output yang dihasilkan dari program di samping?.

A. A A B B

B. A B A B

C. B B A A

D. A B B A

E. B A B A

Question #8

```
class Car:
    name = "Tesla";

    # Constructor
    def __init__(self, name):
        print(f"{self.name}")
        self.name = name;

    # Destructor
    def __del__(self):
        print(f"{self.name}")

# create objects
car1 = Car("Toyota")  # Creating car object 1
car2 = Car("Honda")   # Creating car object 2
```

Perhatikan penerapan constructor dan destructor pada class Car di samping. Apa output yang dihasilkan dari program di samping?.

A. Toyota Honda Honda Toyota

B. Tesla Tesla Toyota Honda

C. Honda Toyota Toyota

D. Toyota Honda

E. Toyota Honda Tesla

Question #9

```python
# parent class
class Animal:
    def __init__(self, name):
        self._name = name

    def make_sound(self):
        pass

# child class
class Dog(Animal):
    def make_sound(self):
        return "Woof!"

# child class
class Cat(Animal):
    def make_sound(self):
        return "Meow!"

# test method
def test_polymorphism():
    animals = [Dog("Buddy"), Cat("Whiskers")]
    for animal in animals:
        print(animal.make_sound())

# call method
test_polymorphism()
```

Apa yang akan dicetak oleh kode berikut?

A. Meow! Meow!

B. Meow! Woof!

C. Woof! Woof!

D. Woof! Meow!

E. Error

Question #10

```python
# Parent Class
class Human:
    def run(self):
        print("Method 1")

    def see(self):
        print("Method 2")

    def __sleep(self):
        print("Method 3")

    def __funny(self):
        print("Method 4")

# Child Class
class Teacher(Human):
    # Method #1
    def teach(self):
        print("I'm teaching !!!");

    # Method #2
    def play(self):
        print("I'm playing !!!");


# create objects
obj1 = Human()
obj2 = Teacher()
```

Perhatikan dengan teliti proses pewarisan di samping. Object dari class Teacher dapat mengakses berapa method?

A. 1 Method

B. 2 Method

C. 3 Method

D. 4 Method

E. 5 Method

END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU